Honor's Project: Gmail Phishing Detection Chrome Extension with Machine and Deep Learning

Evan C. Richard

Missouri University of Science and Technology

May 2026

GitHub Repository: https://github.com/ERichard007/PhishingDetectorChromeExtension.git

Introduction

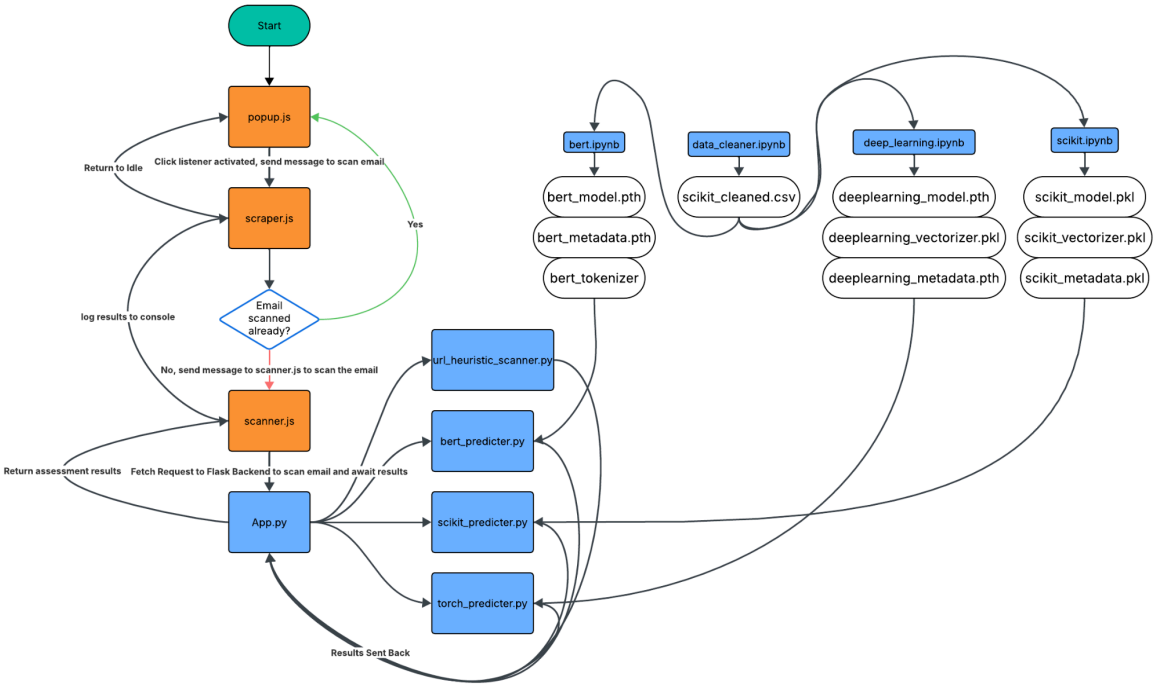
Phishing is one of if not the most common cyber attack of today. It is a growing threat, and a social engineering attack that can be hard to detect. Current solutions to this problem are limited and with the advent of AI, it has blown up to be an even bigger issue. My project attempts to combat this rising problem, by implementing my own AI models in a simple extension for your browser. Ideally this extension could sit idly in the background waiting for you to open an email, and then scan it once you do. Upon scanning the email it will return to you a risk assessment score and an analysis of how it came to its conclusion. I believe something like this could be extremely useful for the average employee at a business, the more elderly people who might be more easily taken advantage of, and even people higher up in businesses who may fall victim to spear phishing or whaling. Ultimately this project develops a machine-learning-powered phishing detection system implemented as a Chrome extension for Gmail that combines multiple AI models and heuristic URL analysis to classify potentially malicious emails.

Background

Traditionally, phishing was detected via signature rules. This could include things like blacklisting, signature scanning, and simply user training. However, the more modern approach to fight against phishing attacks has been machine learning. Machine learning can keep up with the AI automated phishing, by no longer simply relying on heuristics or signatures. A machine learning algorithm can analyze the content and the context of the email to protect the user against a potential attack.

Architecture

So firstly my chrome extension sits in the background. Once you open an email, you can click on the extension and click scan email. This firstly calls my javascript that will retrieve the subject, body, and any attachments in the url. Upon doing this the javascript will make a fetch request to my flask backend. Flask then makes separate request to my url_scanner, bert_predicter, scikit_predicter, and torch_predicter. All of which utilize their respective trained models and vectorizers to make an accurate prediction based on the subject and text of the email that was scraped. They then send back to App.py their results and predictions separately. App.py makes an ensemble prediction based on the accuracies and weights of each of the models, as well as the prediction made by the url heuristics algorithm. This final result is sent back to scanner.js which is then sent back to scraper.js who logs it in the console to be seen. Finally we are returned to idle mode awaiting another scan.



Data Preparation

The dataset I used to train these AI models are the “Seven Phishing Email Datasets” found here https://doi.org/10.6084/m9.figshare.25432108. I utilize the jupyter notebooks file data_clearner.ipynb to remove duplicates, remove empty values, and deal with malformed lines. This leaves me with a cleaned .csv file that I utilize to train my AI models. For this project for each of my models I split the data by 60/20/20. This means 60% of my data is used for training, 20% used for validation, and the last 20% used for testing. The testing data was never revealed to me until the very end to ensure no data leakage or inflated performance results.

Machine Learning

Random Forest + TF-IDF

For the simplest of the three models, the non-deep learning model, I decided to use the random forest machine learning algorithm, along with a TF-IDF vectorizer. I chose TF-IDF simply because it is simple to use and gives fast results. As well as its perfect for what I'm attempting to do, which is classify text as phishing or non-phishing. TF-IDF does this by assigning words in a hierarchy of importance, which is perfect because keywords like "free", "urgent", "now", and so on will be categorized as having higher importance if they appear less frequently and are more distinct. The random forest algorithm I chose because in my research I found that it was the most accurate for my specific problem with classification. I actually began by using the naive-bayes algorithm, but it was giving me very mixed results. Random forest splits the training data into many random trees, and gives me a fairly accurate prediction.

Deep Learning Model

For my main deep learning model, I utilized a couple of different features with PyTorch. This model attempted to learn phishing patterns through multiple neural network layers trained on Bag-Of-Words vectorized email data. Which transformed the text into a vector containing word occurrence frequencies, perfect for my phishing classification that I need. The neural network I used consisted of connected linear layers. I used GELU as my activation function along with Dropout for regularization. GELU is an improvement on ReLU, providing a smoother gradient that often performs better and was recommended to me by a friend in Deep Learning.

Dropout was used in an attempt to reduce overfitting, randomly disabling neurons that force the network to learn rather than recognizing patterns. Finally the results were interpreted using torch.sigmoid during evaluation to produce a single output value. Finally for my loss and optimization I used BCEWithLogitsLoss and Adam. Adam was used in tandem with learning rate and weight decay, which helps train the model fast and more reliably. BCE combines a sigmoid layer and BCELoss into something more numerically stable. When experimenting with hyperparameters, I found that shallow networks actually did the best. Meaning as I increased the width and decreased depth, I got better and better results. Models that were trained on deeper layers seemed to get stuck. My best hypothesis on why this happened was because deeper layers introduced levels of complexity that were too much for the simple data representation and also probably because of my lack of knowledge on the subject to fix it properly. Increased layer width often increased my accuracy on its own, but also increased the amount of training time and computation power. Training was performed on a combination of my GPU and on Google Colab's GPUs. With Google Colab I could only do so many layers wide and deep before the kernel crashed, but on my own GPU I did not experience those limits and could spend more time in the training phase. Overall experimentation showed that often the simpler networks performed much better than the complex ones, and were more computationally efficient for what I was trying to achieve anyway. In the end the results would produce a model.pth, vectorizer.pkl, and metadata.pth that would be used later on in my flask backend.

BERT Transformer Model

Now the most complex of the three models, BERT. I chose BERT as a fun way to introduce a new concept into my project. This model uses contextual language understanding

rather than simple word frequency analysis. This seems perfect for my project, so I wanted to experiment to see if BERT could perform better than the random forest and feedforward neural network approaches. I implemented this firstly by using the pretrained ‘bert-base-uncased’ model I got from Hugging Face. So BERT works by analyzing the contextual relationships between words in both directions of a sentence, which in theory should allow it to capture language patterns like urgency, social engineering tactics, and other suspicious uses of language. To train BERT I again used Adam as my optimizer to increase efficiency and decrease training time as well as used BCEWithLogitsLoss for binary classification again. The model was trained on 3 EPOCHS, mainly because that decreased my loss plenty without risking overfitting as well as just taking way too long to train. Training was done on my personal GPU this time as Google Colab’s GPUs could not handle how computation expensive this was to do. Finally the results of my evaluation outperformed both my other deep learning model and my random forest model. In testing it was also found to be more reliable than both when checking emails for phishing. This is probably likely due to my hypothesis about BERT being great at detecting contextual information, which in turn makes it a great way to detect phishing. Since phishing is largely a social engineering attack, BERT’s ability to learn these language context patterns makes it exceptionally good at detecting potential phishing attacks. After evaluation and testing, model.pth, metadata.pth, and tokenizer were all saved to be used by my Flask backend.

URL Heuristics

So instead of using a machine learning algorithm here, I decided to use simple heuristics to detect possibly malicious URLs. I did this by utilizing a python library urllib.parse. In which I split the given URLs into 5 different parts: the Scheme, Authority, Path, Query and Fragment.

Firstly for the Scheme I look to find any not-http protocols or ones that would be suspicious like ftp. In my authority section I looked for IP addresses, suspicious TLDs, non-standard ports, obfuscation via @, large amounts of subdomains, impersonation attempts, and long lengths. In my path section I looked for malicious keywords, impersonation attempts, long path segments, encoding patterns, malicious extensions like .exe, // obfuscation, and encodings. My query scan consisted of looking for redirection attempts, passwords transferring, long lengths, and encodings. Finally, in my fragment section I looked for malicious keywords again, urls, encodings, and length. To finish it all up I used a popular OSINT tool called Phish-Tank, and utilized their large library of up to date phishing urls to try and match if the current url being scanned is in that active database or not. When analyzing this of course I found the obvious problem, way too many false positives and way too much to tweak. I could've spent days just tweaking every little rule, just to find that a machine learning approach would probably do the job faster and better. Upon completion, the algorithm would send back a normalized number between 0 and 1 that corresponds to how likely all of the urls are related to a phishing attack.

Ensemble Prediction

So in order to combat the issues with a lot of false positives and false negatives, I took a certain approach to combining all of the results from the three different models and the url heuristics. Firstly I gave each model its own weight depending on the accuracy it achieved on its testing phase. This meant that BERT got the highest weight and the Random Forest model got the lowest. I then took these weights and multiplied it by its phishing probability score and added them all together to give me one final number. I then took this number and multiplied it by 0.9 and the url heuristics results by 0.1 and added them both together. Essentially giving the machine

learning algorithms 90% influence over the final results and the url heuristics algorithm just 10% since it was more likely to give random results. The equations can be seen as follows:

W1 = forest accuracy / total models accuracy

W2 = deep learning accuracy / total models accuracy

W3 = bert accuracy / total models accuracy

*Machine learning probability = W1 * Forest Prediction + W2 * Deep Learning Prediction + W3 * Bert Prediction*

*Final Probability = 0.9 * Machine Learning probability + 0.1 * Url Final Score*

I take this number and depending on the threshold it meets, I assign it a threat rating "High", "Medium", "Low", and None. This is to combat the simple problem of assigning the final result as a 0 or 1 or phishing and non-phishing. Why I did this will be explained more in depth below.

Experimental Results

Random Forest

The confusion matrix and PRF table for my random forest machine learning model can be seen below. For the confusion matrix the following is how the numbers correspond:

True Negative	False Positive

False Negative	True Positive

Predicted Legitimate and were Legitimate	Predicted Phishing and were Legitimate
Predicted Legitimate and were Phishing	Predicted Phishing and were Phishing

These numbers also will continue to correspond to the confusion matrix of the Deep Learning and BERT model.

Confusion Matrix For 3 Models

Random Forest Model	
7794	58
303	7363
Deep Learning Model	
7676	251
258	7333
BERT Model	
7815	22

86	7595

Precision, Recall, and F1 Table For 3 Models

Model	Validation Accuracy	Test Accuracy	Precision	Recall	F1-Score
Random Forest + TF-IDF	0.9745	0.9767	0.98	0.98	0.98
Neural Network	0.9812	0.9672	0.97	0.97	0.97
BERT Transformer	0.9930	0.9930	0.99	0.99	0.99

Overall

So looking at the confusion matrix, we get results that follow with my evaluation. BERT got the least amount of false positives and false negatives on the testing data. Surprisingly however, the deep learning model did worse than both the random forest model and BERT model. We can also see this in the classification report where the test accuracy was the worst of the three but the validation accuracy was only the 2nd worst. I hypothesize that this was caused

by a lack of training actually and putting more time into the training phase might fix these issues, this also points to possible overfitting of our model in which we were beginning to recognize patterns instead of making predictions. Maybe I could've fixed this by configuring some hyperparameters like dropout to fix this. Also I notice in our confusion matrix that the models tended to predict something was legitimate when it was actually phishing. This actually follows what I found in practice, to combat this I introduce the ensemble prediction method mentioned before and it changed to most things actually being deemed as phishing.

Limitations

There are some limitations I would like to acknowledge with this project. Firstly the dataset I used came from a website that had already curated a bunch of email to either be phishing or non-phishing. This works fine for this small project, but for a real world implementation this is something that would have to constantly be addressed and fixed. As the world of phishing is constantly changing. Also my models may fail if exposed to something novel that doesn't match the trained data.

Secondly my AI models seem to trend towards labeling things as phishing, leading to situations in which they label totally legitimate emails as phishing. To combat this in my project, I added the ensemble system and had it more or less predicted based on a spectrum from None to High rather than just a 0 or 1. This was a haphazard way of fixing a system that trends toward false positives.

Third, the URL and heuristic based algorithm for checking the attachments is inherently flawed and would require constant upkeep and tweaking in order to actually make it work. Just incorporating them into some kind of machine learning algorithm would be greatly more

efficient than what I chose to do. It is also only really good at catching things already known to us, any new novel ideas would not be captured at all.

Fourth, computationally these models (especially BERT) took a long time to train and my hardware was not quite up to spec in order to handle really large datasets and limits the depth of my experimentation. Since I need to retrain a model everytime I want to tweak it, this takes a long time if I want to change a couple of hyperparameters. Which restricted the number of epochs and configurations I could try.

Finally, the biggest limitation is myself. This is my first big project in this area of computer science and was the first time I really did any large amount of research into artificial intelligence. I was aided by a friend who was taking a class in deep learning to help guide me on how some things work and what would work best with what. Overall though, when things got tough it was my lack of knowledge in the area that really stunted any forward progress, but this is something I'm hoping to progress on in the future and the upcoming semesters of school.

Conclusion

This project explored the development and implementation of AI as a phishing detection system. I chose to implement it as a chrome extension for ease of use, or to simulate how someone in the real world would use it. Because to be honest, the people who really need to be warned about a phishing attack are probably not going to take the time to check each and every email manually through some kind of service. So ideally we need something automatic, fast, reliable, easy to use, and understand for someone who is not tech savvy. Most phishing attacks happen on people just like that. I think this project really showcased the importance of just how tweaked these models would have to be, because in testing a false negative is fine. However, in

the real world a false negative could mean successful infiltration, infection, and exfiltration of important data. Overall, this project successfully created a phish detection system using AI models and provided due insight on just how useful AI can be as a modern technique for identifying a phishing attack.